Assignment 16.4

Name: **Navya Revuri**

Batch:44

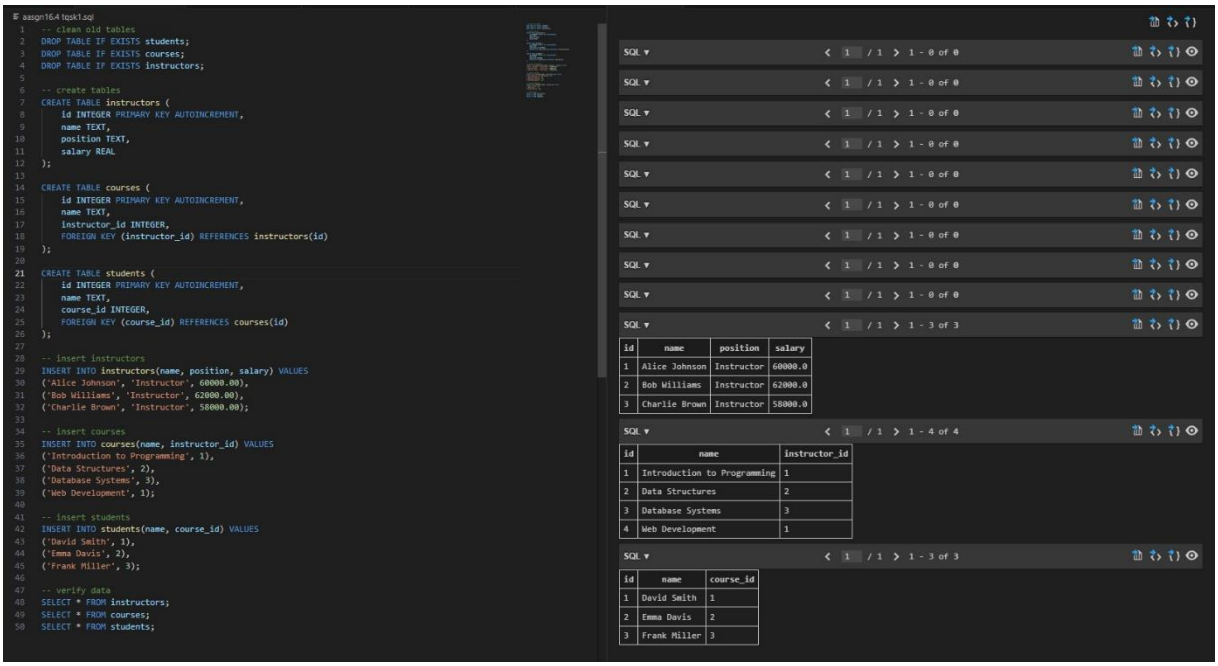
HNO: 2303A52483

Task 1: Database Schema Design for an Online Course Platform

Prompt:

#Generate SQL INSERT statements to populate a database with sample data. The data should include at least 3 instructors, 4 courses, and 3 students. Ensure the data is realistic and properly structured. Also include SELECT queries to verify that the data has been inserted correctly.

Code:



```sql
-- clean old tables
DROP TABLE IF EXISTS students;
DROP TABLE IF EXISTS courses;
DROP TABLE IF EXISTS instructors;

-- create tables
CREATE TABLE instructors (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    position TEXT,
    salary REAL
);

CREATE TABLE courses (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    instructor_id INTEGER,
    FOREIGN KEY (instructor_id) REFERENCES instructors(id)
);

CREATE TABLE students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    course_id INTEGER,
    FOREIGN KEY (course_id) REFERENCES courses(id)
);

-- insert instructors
INSERT INTO instructors(name, position, salary) VALUES
('Alice Johnson', 'Instructor', 60000.00),
('Bob Williams', 'Instructor', 62000.00),
('Charlie Brown', 'Instructor', 58000.00);

-- insert courses
INSERT INTO courses(name, instructor_id) VALUES
('Introduction to Programming', 1),
('Data Structures', 2),
('Database Systems', 3),
('Web Development', 1);

-- insert students
INSERT INTO students(name, course_id) VALUES
('David Smith', 1),
('Emma Davis', 2),
('Frank Miller', 3);

-- verify data
SELECT * FROM instructors;
SELECT * FROM courses;
SELECT * FROM students;
```

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 3 of 3

id	name	position	salary
1	Alice Johnson	Instructor	60000.0
2	Bob Williams	Instructor	62000.0
3	Charlie Brown	Instructor	58000.0

SQL ▾ 1 / 1 1 - 4 of 4

id	name	instructor_id
1	Introduction to Programming	1
2	Data Structures	2
3	Database Systems	3
4	Web Development	1

SQL ▾ 1 / 1 1 - 3 of 3

id	name	course_id
1	David Smith	1
2	Emma Davis	2
3	Frank Miller	3

Observation:

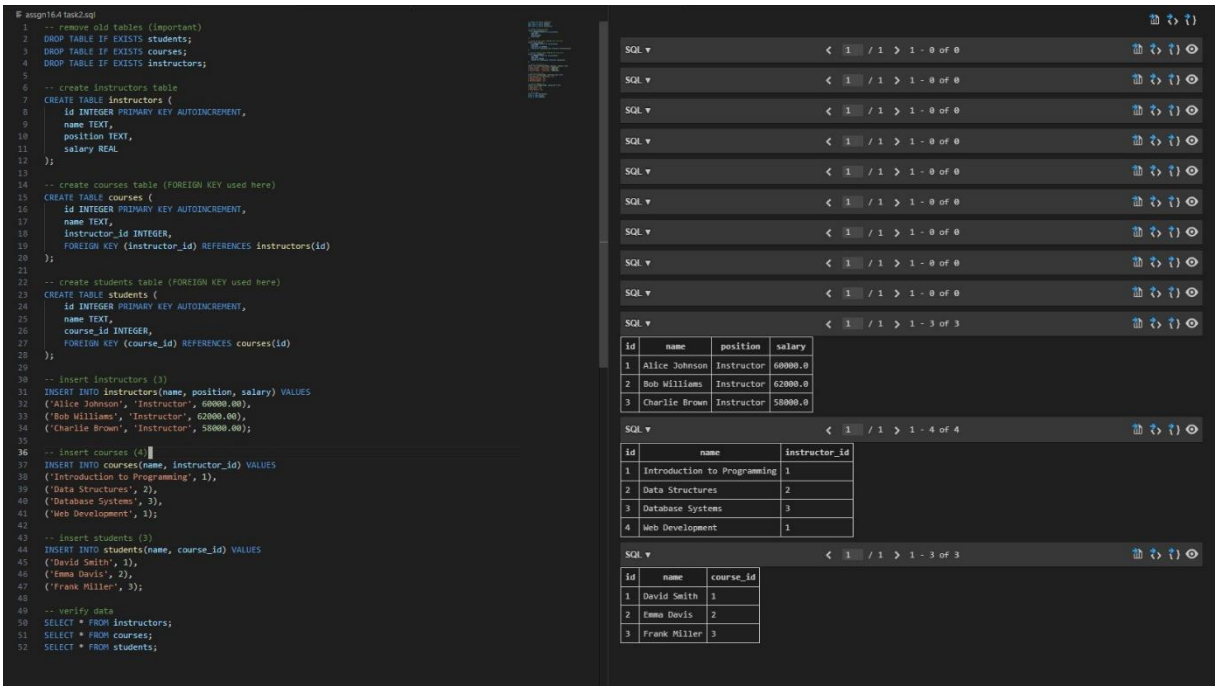
The SQL INSERT statements were successfully executed without errors in SQLite. Three instructors, four courses, and three students were added to their respective tables. The SELECT queries confirmed that all records were correctly inserted and displayed as expected. The relationships between courses and instructors were maintained using instructor_id.

Task 2: Populate the Database with Sample Records

Prompt:

Generate SQL INSERT statements to populate a database with sample records. The database should include at least 3 instructors, 4 courses, and 3 students. Ensure proper relationships between tables and include SELECT queries to verify that the data has been inserted correctly.

Code:



```sql
-- remove old tables (important)
DROP TABLE IF EXISTS students;
DROP TABLE IF EXISTS courses;
DROP TABLE IF EXISTS instructors;

-- create instructors table
CREATE TABLE instructors (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    position TEXT,
    salary REAL
);

-- create courses table (FOREIGN KEY used here)
CREATE TABLE courses (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    instructor_id INTEGER,
    FOREIGN KEY (instructor_id) REFERENCES instructors(id)
);

-- create students table (FOREIGN KEY used here)
CREATE TABLE students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    course_id INTEGER,
    FOREIGN KEY (course_id) REFERENCES courses(id)
);

-- insert instructors (3)
INSERT INTO instructors(name, position, salary) VALUES
('Alice Johnson', 'Instructor', 60000.00),
('Bob Williams', 'Instructor', 62000.00),
('Charlie Brown', 'Instructor', 58000.00);

-- insert courses (4)
INSERT INTO courses(name, instructor_id) VALUES
('Introduction to Programming', 1),
('Data Structures', 2),
('Database Systems', 3),
('Web Development', 1);

-- insert students (3)
INSERT INTO students(name, course_id) VALUES
('David Smith', 1),
('Emma Davis', 2),
('Frank Miller', 3);

-- verify data
SELECT * FROM instructors;
SELECT * FROM courses;
SELECT * FROM students;
```

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 3 of 3

id	name	position	salary
1	Alice Johnson	Instructor	60000.0
2	Bob Williams	Instructor	62000.0
3	Charlie Brown	Instructor	58000.0

SQL ▾ 1 / 1 1 - 4 of 4

id	name	instructor_id
1	Introduction to Programming	1
2	Data Structures	2
3	Database Systems	3
4	Web Development	1

SQL ▾ 1 / 1 1 - 3 of 3

id	name	course_id
1	David Smith	1
2	Emma Davis	2
3	Frank Miller	3

Observation:

The SQL INSERT statements were successfully executed to populate the database with sample records. Three instructors, four courses, and three students were added. The SELECT queries were used to verify the inserted data, and all records were displayed correctly without any errors. The relationships between instructors, courses, and students were maintained using foreign keys.

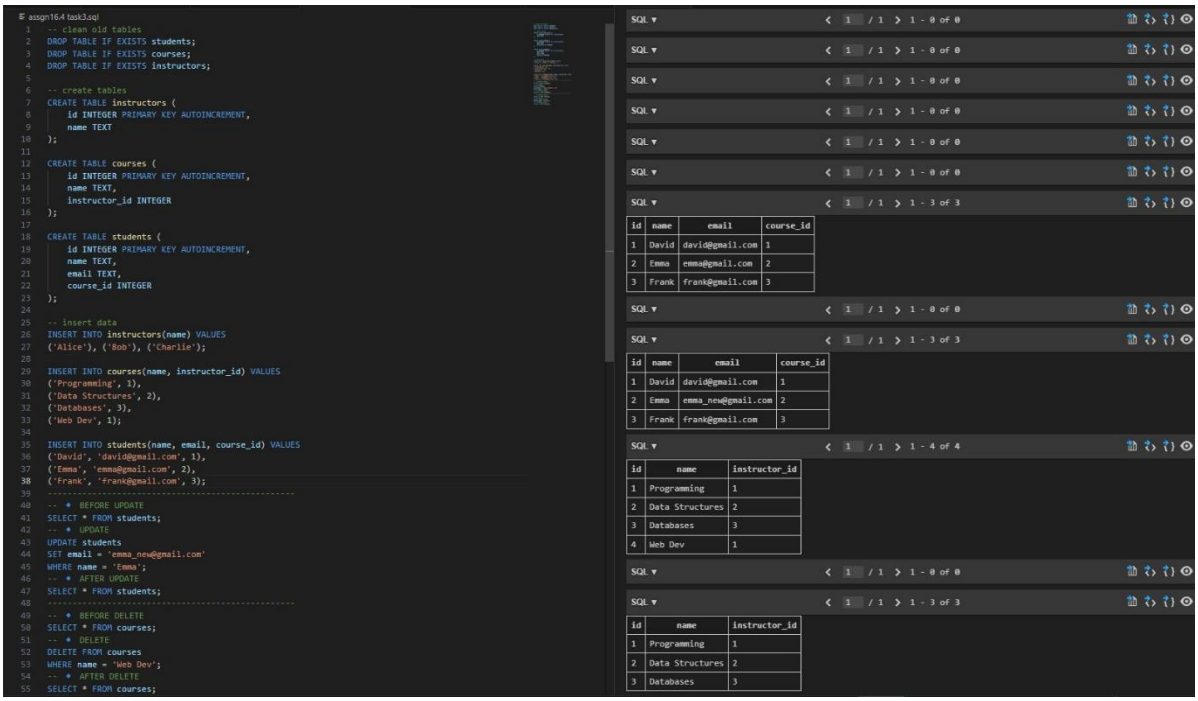
Task 3: Implement Core Database Operations (CRUD)

Prompt:

Generate SQL queries to perform CRUD operations on a database.

The operations should include retrieving all courses, updating a student's email address, deleting a course, and verifying each operation using SELECT queries to ensure correctness.

Code:



```sql
-- clean old tables
DROP TABLE IF EXISTS students;
DROP TABLE IF EXISTS courses;
DROP TABLE IF EXISTS instructors;

-- create tables
CREATE TABLE instructors (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT
);

CREATE TABLE courses (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    instructor_id INTEGER
);

CREATE TABLE students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT,
    course_id INTEGER
);

-- insert data
INSERT INTO instructors(name) VALUES
('Alice'), ('Bob'), ('Charlie');

INSERT INTO courses(name, instructor_id) VALUES
('Programming', 1),
('Data Structures', 2),
('Databases', 3),
('Web Dev', 1);

INSERT INTO students(name, email, course_id) VALUES
('David', 'david@gmail.com', 1),
('Emma', 'emma@gmail.com', 2),
('Frank', 'frank@gmail.com', 3);
-----------------------------------------------
-- * BEFORE UPDATE
SELECT * FROM students;
-- * UPDATE
UPDATE students
SET email = 'emma_new@gmail.com'
WHERE name = 'Emma';
-- * AFTER UPDATE
SELECT * FROM students;
-----------------------------------------------
-- * BEFORE DELETE
SELECT * FROM courses;
-- * DELETE
DELETE FROM courses
WHERE name = 'Web Dev';
-- * AFTER DELETE
SELECT * FROM courses;
```

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 3 of 3

id	name	email	course_id
1	David	david@gmail.com	1
2	Emma	emma@gmail.com	2
3	Frank	frank@gmail.com	3

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 3 of 3

id	name	email	course_id
1	David	david@gmail.com	1
2	Emma	emma_new@gmail.com	2
3	Frank	frank@gmail.com	3

SQL ▾ 1 / 1 1 - 4 of 4

id	name	instructor_id
1	Programming	1
2	Data Structures	2
3	Databases	3
4	Web Dev	1

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 3 of 3

id	name	instructor_id
1	Programming	1
2	Data Structures	2
3	Databases	3

Observation:

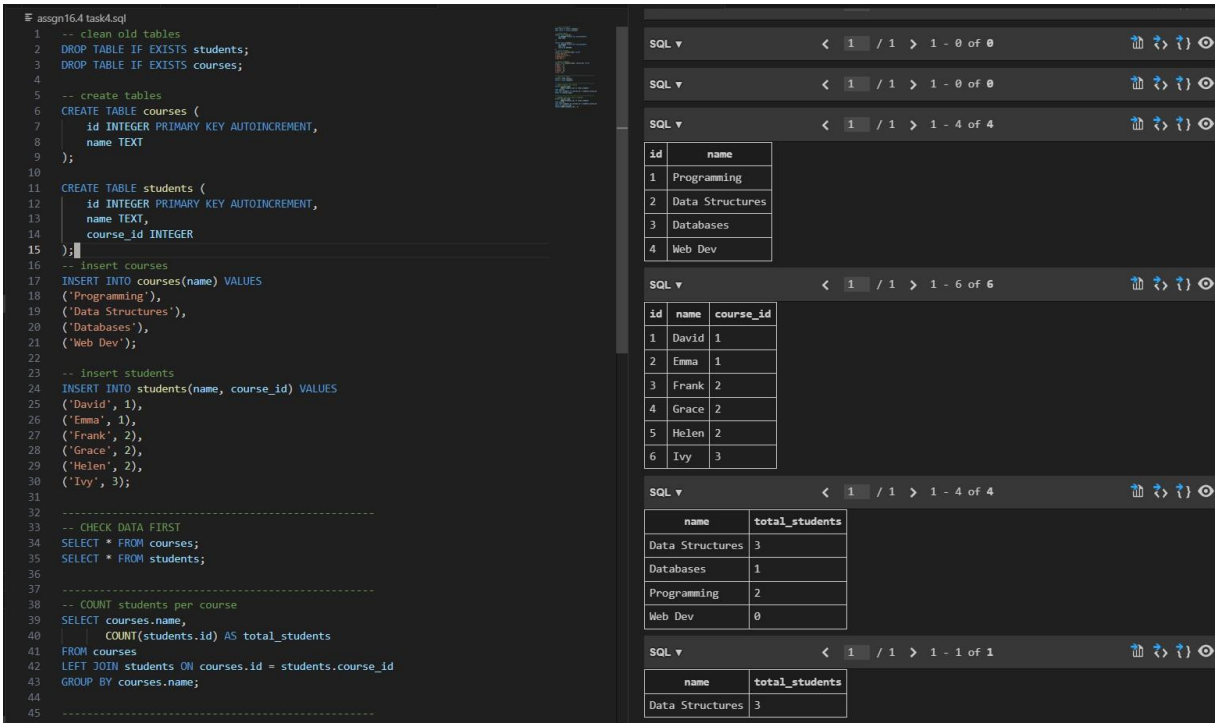
The CRUD operations were successfully performed on the database. The SELECT query retrieved all course records. The UPDATE operation modified a student's email address correctly, which was verified using a SELECT query. The DELETE operation removed a course from the database, and the change was confirmed by querying the courses table. No unintended data loss occurred during these operations.

Task 4: Reporting Using Aggregate Queries

Prompt:

Generate SQL queries to produce summary reports using aggregate functions. The queries should count the number of students enrolled in each course and list courses that have more than a specified number of enrollments. Use GROUP BY and HAVING clauses appropriately and verify the correctness of the results.

Code:



```sql
-- clean old tables
DROP TABLE IF EXISTS students;
DROP TABLE IF EXISTS courses;

-- create tables
CREATE TABLE courses (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT
);

CREATE TABLE students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    course_id INTEGER
);
-- insert courses
INSERT INTO courses(name) VALUES
('Programming'),
('Data Structures'),
('Databases'),
('Web Dev');

-- insert students
INSERT INTO students(name, course_id) VALUES
('David', 1),
('Emma', 1),
('Frank', 2),
('Grace', 2),
('Helen', 2),
('Ivy', 3);

-- CHECK DATA FIRST
SELECT * FROM courses;
SELECT * FROM students;

-- COUNT students per course
SELECT courses.name,
       COUNT(students.id) AS total_students
FROM courses
LEFT JOIN students ON courses.id = students.course_id
GROUP BY courses.name;
```

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 4 of 4

id	name
1	Programming
2	Data Structures
3	Databases
4	Web Dev

SQL ▾ 1 / 1 1 - 6 of 6

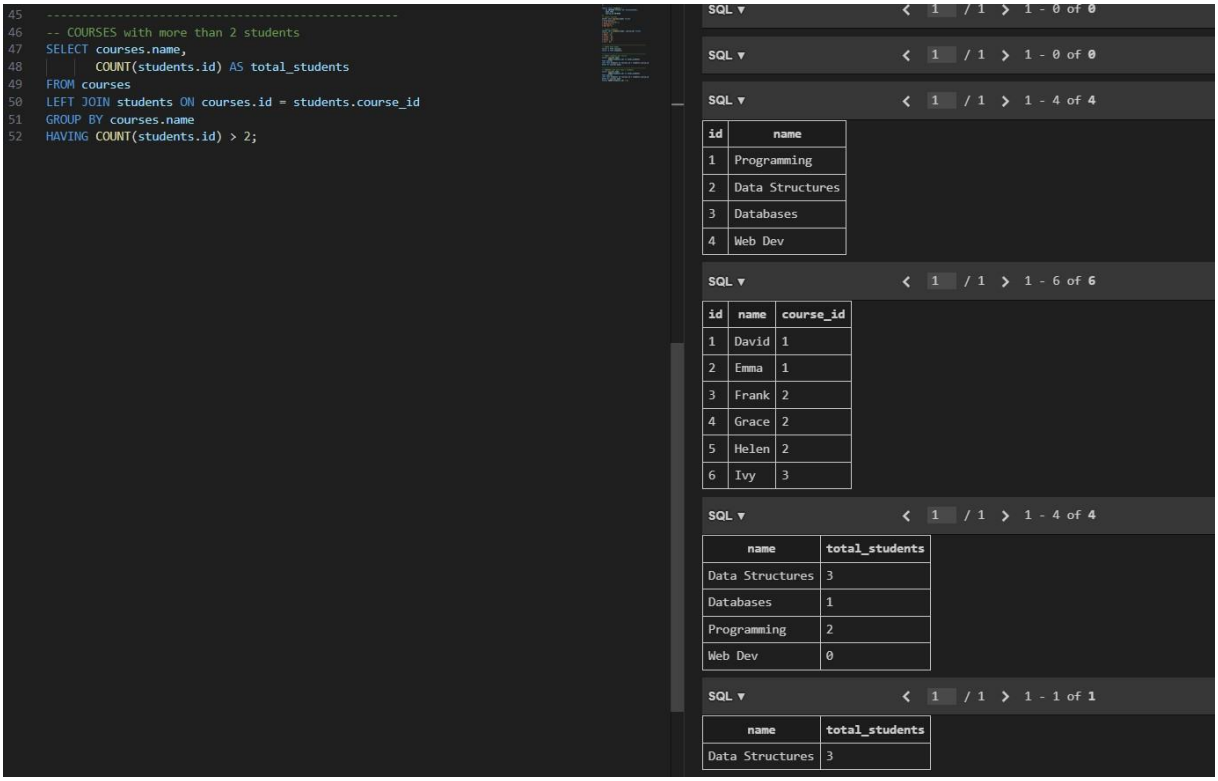
id	name	course_id
1	David	1
2	Emma	1
3	Frank	2
4	Grace	2
5	Helen	2
6	Ivy	3

SQL ▾ 1 / 1 1 - 4 of 4

name	total_students
Data Structures	3
Databases	1
Programming	2
Web Dev	0

SQL ▾ 1 / 1 1 - 1 of 1

name	total_students
Data Structures	3



```sql
-- COURSES with more than 2 students
SELECT courses.name,
       COUNT(students.id) AS total_students
FROM courses
LEFT JOIN students ON courses.id = students.course_id
GROUP BY courses.name
HAVING COUNT(students.id) > 2;
```

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 4 of 4

id	name
1	Programming
2	Data Structures
3	Databases
4	Web Dev

SQL ▾ 1 / 1 1 - 6 of 6

id	name	course_id
1	David	1
2	Emma	1
3	Frank	2
4	Grace	2
5	Helen	2
6	Ivy	3

SQL ▾ 1 / 1 1 - 4 of 4

name	total_students
Data Structures	3
Databases	1
Programming	2
Web Dev	0

SQL ▾ 1 / 1 1 - 1 of 1

name	total_students
Data Structures	3

Observation:

Aggregate queries were successfully used to generate summary reports. The GROUP BY clause was used to group students by course, and the COUNT function calculated the number of students in each

course. The HAVING clause was applied to filter courses with more than a specified number of enrollments. The results were accurate and matched the inserted data.

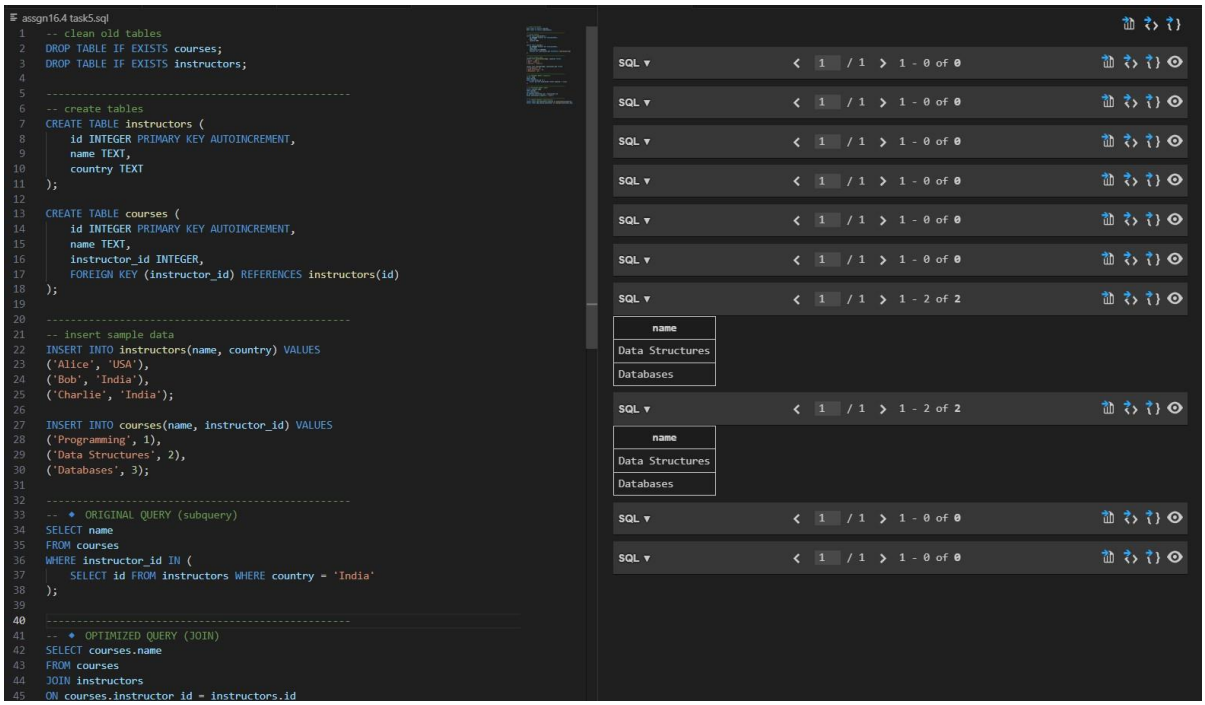
Task 5: Query Optimization with AI Suggestions

Prompt:

Analyze an existing SQL query that retrieves courses taught by instructors from a specific country. Refactor the query using efficient joins and suggest indexing techniques to improve performance.

Compare the results before and after optimization to ensure correctness and improved readability.

Code:



```
assgn16.4 task5.sql
-- clean old tables
DROP TABLE IF EXISTS courses;
DROP TABLE IF EXISTS instructors;

-------------------------------------------------
-- create tables
CREATE TABLE instructors (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    country TEXT
);

CREATE TABLE courses (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    instructor_id INTEGER,
    FOREIGN KEY (instructor_id) REFERENCES instructors(id)
);

-------------------------------------------------
-- insert sample data
INSERT INTO instructors(name, country) VALUES
('Alice', 'USA'),
('Bob', 'India'),
('Charlie', 'India');

INSERT INTO courses(name, instructor_id) VALUES
('Programming', 1),
('Data Structures', 2),
('Databases', 3);

-------------------------------------------------
-- 🔹 ORIGINAL QUERY (subquery)
SELECT name
FROM courses
WHERE instructor_id IN (
    SELECT id FROM instructors WHERE country = 'India'
);

-------------------------------------------------
-- 🔹 OPTIMIZED QUERY (JOIN)
SELECT courses.name
FROM courses
JOIN instructors
ON courses.instructor_id = instructors.id
```

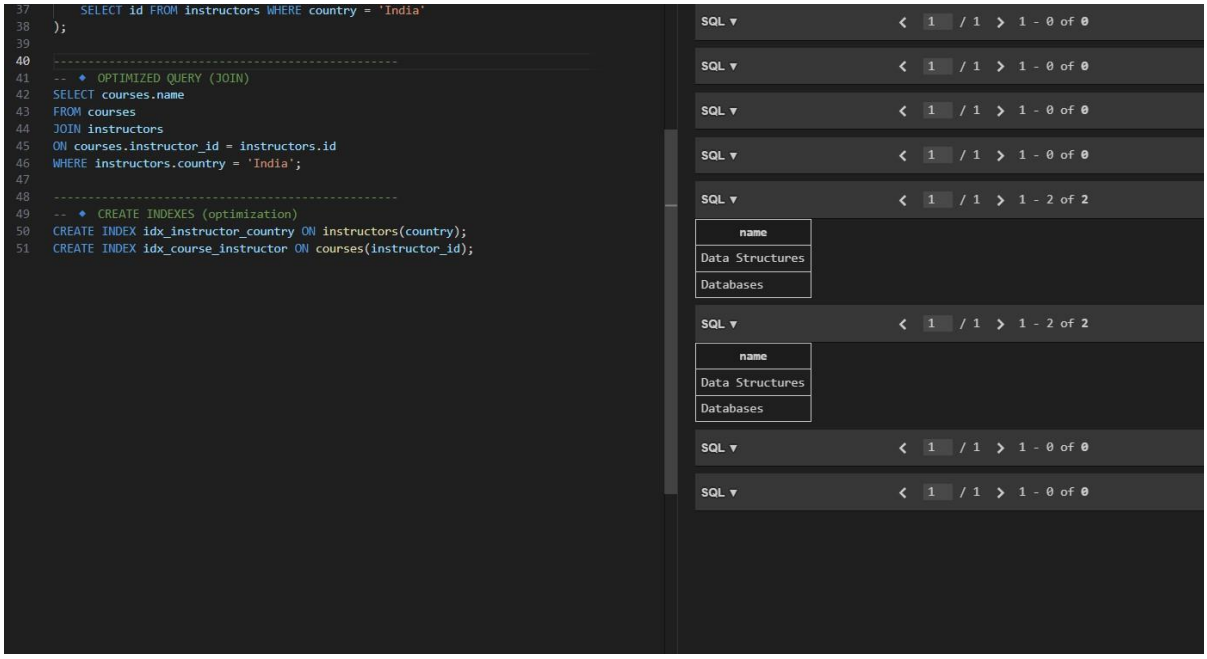
```
SQL ▾  1 / 1  1 - 0 of 0
SQL ▾  1 / 1  1 - 0 of 0
SQL ▾  1 / 1  1 - 0 of 0
SQL ▾  1 / 1  1 - 0 of 0
SQL ▾  1 / 1  1 - 0 of 0
SQL ▾  1 / 1  1 - 0 of 0
SQL ▾  1 / 1  1 - 2 of 2
```

name
Data Structures
Databases

```
SQL ▾  1 / 1  1 - 2 of 2
```

name
Data Structures
Databases

```
SQL ▾  1 / 1  1 - 0 of 0
SQL ▾  1 / 1  1 - 0 of 0
```



Observation:

The original query used a subquery to retrieve courses based on instructor country, which can be inefficient for large datasets. The optimized query replaced the subquery with a JOIN, improving readability and performance. Indexes were suggested on frequently used columns such as country and instructor_ id to further enhance query speed. Both queries produced identical results, confirming correctness after optimization.